UNIVERSITE DE RENNES 1
UFR Mathématiques **2026-2027**

Programmation objet - C++

TP 4

À la fin de chaque créneau horaire, rendre à l'enseignant un résumé de 10 lignes sur feuille racontant ce qui a été fait pendant le créneau.

1. Dans le fichier `matrice1.h`, écrire une classe `Matrice` dont les données sont le nombre de lignes `nbl` et le nombre de colonnes `nbc` de la matrice, et un pointeur `double **val` permettant d'accéder aux valeurs de ses éléments. Écrire le constructeur `Matrice (int nl, int nc)` qui alloue la mémoire nécessaire au stockage des valeurs, chaque ligne de la matrice étant un tableau adressé via un pointeur lui-même élément d'un tableau de pointeurs. Écrire une fonction membre `void affiche()` qui affiche le contenu de la matrice. Écrire une fonction membre `void init (double r)` qui initialise les éléments de la matrice à la valeur `r`. Tester.
Nota : Utiliser le destructeur par défaut jusqu'à mention contraire.

2. Surcharger l'opérateur d'addition de façon que l'instruction `C = A+B;` réalise l'addition des deux matrices `A` et `B` et stocke le résultat dans `C`. Le prototype de la fonction sera :

```
Matrice operator+(Matrice&);
```

Nota : Il sera vérifié que les tailles sont compatibles.

⟶ Recopier le fichier `matrice1.h` dans `matrice2.h` et référencer désormais celui-ci dans la suite.

3. **a)** Écrire le destructeur de cette classe de manière à libérer correctement la mémoire allouée.

b) Tester sur un programme qui contient l'instruction `Matrice C(A);`, puis les instructions `Matrice D; D=A+B;`. Où se situe la différence ? Expliquer la nature du problème avant de passer à la question suivante.

⟶ Recopier le fichier `matrice2.h` dans `matrice3.h` et référencer désormais celui-ci dans la suite.

4. **a)** Faire en sorte que le programme fonctionne. Lorsque c'est fait, identifier ce qui se passe lors de l'instruction `Matrice E=B+A;`.

b) Tester l'associativité avec l'instruction `Matrice F=A+B+A;` qui est équivalente à l'instruction `Matrice F=(A+B)+A;` . Par contre, l'instruction `Matrice G=A+(B+A);` provoque une erreur de compilation (à condition que le prototype de l'opérateur d'addition soit toujours celui de la question 2). Le constater.
En conséquence, dans la suite, utiliser le prototype `Matrice operator+(const Matrice&);`

⟶ Recopier le fichier `matrice3.h` dans `matrice4.h` et référencer désormais celui-ci dans la suite.

5. On souhaite transformer la classe `Matrice` pour en faire une classe abstraite dont dérivent les classes `MatSym` et `MatFull`, cette dernière correspondant au stockage "plein" déjà mis en œuvre, la classe `MatSym` correspondant au cas d'une matrice symétrique avec le stockage exclusif de la partie triangulaire inférieure.

a) Dans un premier temps, mettre en œuvre la classe de base `Matrice` et la classe dérivée `MatFull` en reprenant ce qui a déjà été fait. Identifier parmi les fonctions membres déjà écrites celles qui peuvent être rattachées à la classe de base et celles qui sont spécifiques à un type particulier de matrice.

b) Faire des fonctions `affiche` et `init` des fonctions virtuelles pures, qui sont adaptées à la nature de la matrice (pleine ou symétrique).

c) Ajouter la classe dérivée `MatSym` en faisant en sorte que seule la partie triangulaire inférieure soit stockée. Écrire notamment les constructeurs `MatSym (int n)` et `MatSym (const MatSym&)`, ainsi que les fonctions `affiche` et `init`.

Tester ces différentes fonctionalités en s'inspirant du programme `q5.cpp` situé dans le répertoire `tp4` de l'espace Moodle associé à ce module.

Nota : Les opérations réalisées jusqu'à présent font intervenir des matrices de même type. Pour permettre l'addition de deux matrices indépendamment de leur nature, il faudrait surcharger les opérateurs d'addition et d'affectation en prévoyant tous les cas possibles. La classe `Matrice` *proposée ici est criticable mais a seulement pour but d'expérimenter rapidement les notions exposées en cours.*